

Изпитно задание:

Контейнеризация и оркестрация с Docker Compose

Цел

Създайте локална многосървисна среда с Docker, която демонстрира:

- публично достъпно приложение, скалирано с повече от 1 инстанция; • вътрешно приложение в защитена мрежа (без публичен порт);
- запазване (persist) на логове чрез volume;
- базов image и отделни Dockerfile-и за всяка услуга (service) с използване на ARG и ENV;
- оркестрация с docker-compose.yml.

Език за имплементация: свободен избор (напр. TS/JS, Python, Go,

Java и др.). Оркестрация: само Docker Compose!

Изисквания към архитектурата

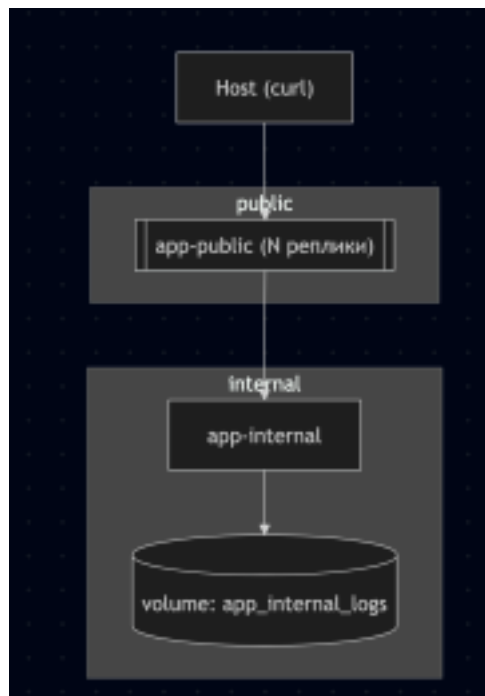
- Минимум 2 приложения:
 - Приложение 1 (публично): приема HTTP заявки на localhost:<ваш_порт>, има повече от 1 инстанция (скалиране).
 - Приложение 2 (вътрешно): работи само в защитена вътрешна мрежа, без публикуван порт; получава заявка от Приложение 1 за частична обработка на резултат.
- Мрежи:
 - public (достъпна от хост само за публичното приложение);
 - internal (споделена между сървисите).

- Логове на Приложение 2 се съхранява (persists) чрез именуван volume, свързан (mounted) в директория с логове в контейнера.
- Скалируемост: демонстрирайте 2+ реплики на публичното приложение чрез `docker compose up --scale <service>=N`.

Примерни архитектури

Може да избирате от два варианта:

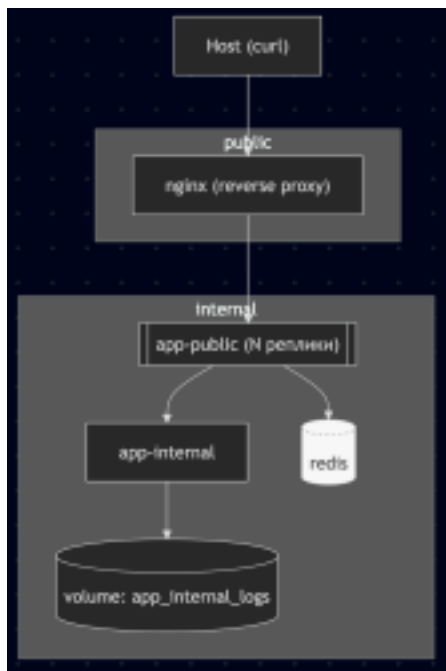
Вариант А: Проста (2 услуги)



- Компоненти:
 - app-public: публично API (напр. GET /process?input=...); ENV PORT=8080, ARG NODE_VERSION; базира се на local/base:1.0.
 - app-internal: вътрешно API (напр. GET /partial?input=...), записва логове в /var/log/app; именуван volume app_internal_logs.
- Мрежи:
 - public: само app-public.

- internal: app-public и app-internal.
- Портове:
 - Публикуван порт само за app-public (8080:8080); app-internal няма публичен порт.
- Healthcheck:
 - За app-internal (напр. GET /health), за да може app-public да изчака готовност.
- Диаграма на потока:

Вариант В: Сложна (reverse проху + кеш)



- Компоненти:
 - nginx: публична входна точка; публикува порт 8080:80; reverse проху към app public с DNS резолюция за реплики.
 - app-public: без публикуван порт; 2+ реплики; извиква app-internal и (по избор) Redis.
 - app-internal: вътрешно API; logs volume.

- redis (по избор): кеширане на междинни резултати.
- Мрежи:
 - public: само nginx.
 - internal: всички услуги.
- Портове:
 - Публикуван порт само за nginx (8080:80).
- Диаграма на потока:

Бележки:

- Външният трафик преминава само през nginx, което позволява коректен баланс към множество реплики на app-public в локална Compose среда.
- app-internal остава непубликуван; достъпът е само през вътрешната мрежа.
- Логовете на app-internal се персистират в именувания volume и се запазват след рестарт.

Функционални изисквания (минимум)

- Създайте базовия image от base/Dockerfile.
- В Dockerfile на всеки сървис използвайте ARG и ENV.
 - Приложение 1 (public): HTTP endpoint (напр. GET /process?input=...) приема заявка от потребител и извиква Приложение 2 през вътрешната мрежа (например http://app internal:PORT/partial). Комбинира/форматира отговора и връща резултат на клиента.
- Приложение 2 (internal): HTTP endpoint за частична обработка (напр. нормализиране/валидиране/агрегиране) и записва логове в монтираната директория.

Контейнерни изисквания

- Базов image:

- Създайте отделен базов image (напр. в base/Dockerfile), който съдържа общи зависимости/настройки;
- В Dockerfile-ите на сървисите използвайте FROM <ваш-базов-image>:<tag>. • Всеки сървис има собствен Dockerfile и показва използване на ARG (напр. версия на рънтайм/билд флаг) и ENV (напр. PORT, RUNTIME_ENV).
- docker-compose.yml:
 - Дефинира двата сървиса, мрежите public и internal, volume за логовете на Приложение 2;
 - Публикува порт само за Приложение 1;
 - Конфигурира зависимости така, че Приложение 1 да изчаква Приложение 2 или и други контейнери (healthcheck е плюс).

За напреднали (по избор)

- Добавете message broker (напр. RabbitMQ/Kafka) и/или Redis кеш. •
- Демонстрирайте взаимодействие (напр. опашка за задачи между Приложение 1 и 2, кеширане на междинни резултати в Redis).

Очаквана структура (пример)

- base/
 - Dockerfile (базовият image)
- app-public/
 - изходен код + Dockerfile
- app-internal/
 - изходен код + Dockerfile (писане на логове в /var/log/app или

подобно) • docker-compose.yml

• README.md

Инструкции за пускане (които вие описвате в README)

- Създаване на базовия image:
- Създаване и старт на средата:
- Мащабируемост (пример с 2 реплики на публичния сървис):
- Проверка на персистирани логове:
- Спиране и почистване:

Изисквания към README.md

- Кратка архитектурна диаграма или описание на потока на заявките (public → internal);
- Използвани images, ARG/ENV параметри и тяхната роля;
- Мрежи, портове, volume-и (кой за какво служи);
- Стъпки за build и старт, мащабиране, тестови примери;
- Как се валидира, че Приложение 2 не е публично достъпно;
- Ако има брокер/Redis: как се демонстрира употребата им.

Предаване

- Всеки студент създава GitLab проект (хранилище) в ФМИ:
<https://git.fmi.uni-plovdiv.bg/>.
- Репото трябва да съдържа целия код, Dockerfile-и, docker compose.yml и README.md.
- Добавете адресът на проекта в classroom.

Условия и ограничения

- Не се изисква сложна бизнес логика — фокусът е контейнеризацията и оркестрацията.
- Може да използвате публични базови images, но ваш базов image трябва да е създаден от вас и използван от сървисите.

Успех!